

Unit 4

Unit 4: Node.js, Express.js, MongoDB — Practical/Oral Exam Notes

Context

Part of [Web Application Development](#) | Practical + Oral Exam Prep | SPPU Third Year IT

High-Yield Oral Topics

- **Node.js Event Loop** — Non-blocking I/O explanation
- **Express Middleware** — What it is, how it works
- **REST API CRUD** — GET/POST/PUT/DELETE with routes
- **MongoDB vs SQL** — Always asked
- **Mongoose Schema** — How to define and use models
- **Full CRUD flow** — Frontend → API → DB → Response

MINI PROJECT: Student Management System — Full Backend

Complete REST API: Node.js + Express + MongoDB for managing students.

SECTION 1 — NODE.JS

Exam Definition

Node.js is an **open-source, cross-platform JavaScript runtime environment** that executes JavaScript code **outside the browser**, built on Chrome's **V8 engine**. It uses an **event-driven, non-blocking I/O model** for high scalability.

Non-Blocking I/O — Viva Explanation

"Traditional servers (Apache) create a new **thread** per request. If 1000 users hit the server, 1000 threads are created — expensive. Node.js uses **ONE thread** and the **Event Loop**. When a slow operation (DB query, file read) is requested, Node doesn't wait — it registers a callback and moves on. When the operation completes, the callback runs. This is **non-blocking I/O** and allows handling thousands of concurrent connections efficiently."

SMS: Node.js Project Setup

```
# 1. Create project folder
mkdir sms-backend
```

```
cd sms-backend
```

```
# 2. Initialize Node project
```

```
npm init -y # Creates package.json
```

```
# 3. Install dependencies
```

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
```

```
# 4. Install dev dependency
```

```
npm install --save-dev nodemon
```

```
# package.json scripts
```

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

Node.js Built-in Modules

```
// File System
```

```
const fs = require('fs');
```

```
fs.readFile('students.json', 'utf8', (err, data) => {  
  if (err) throw err;  
  console.log(JSON.parse(data));  
});
```

```
// HTTP module (raw server – without Express)
```

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'application/json' });  
  res.end(JSON.stringify({ message: 'SMS API Running' }));  
});  
server.listen(3000, () => console.log('Server on port 3000'));
```

```
// Path module
```

```
const path = require('path');
```

```
const filePath = path.join(__dirname, 'public', 'index.html');
```

```
// Events module
```

```
const EventEmitter = require('events');
```

```
const emitter = new EventEmitter();
```

```
emitter.on('studentAdded', (name) => console.log(`${name} added`));
```

```
emitter.emit('studentAdded', 'Rahul');
```

SECTION 2 — EXPRESS.JS

📌 Exam Definition

Express.js is a **minimal, unopinionated web framework for Node.js** that provides routing, middleware support, and utilities for building web applications and REST APIs.

💻 Complete SMS REST API Server

```
// server.js – Main entry point
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

// ===== MIDDLEWARE =====
app.use(cors()); // Allow cross-origin requests (Angular frontend)
app.use(express.json()); // Parse JSON request bodies
app.use(express.urlencoded({ extended: true })); // Parse URL-encoded data

// Request logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} – ${new Date().toISOString()}`);
  next(); // MUST call next() or request hangs
});

// Static files
app.use(express.static('public'));

// ===== DATABASE CONNECTION =====
mongoose.connect(process.env.MONGO_URI || 'mongodb://localhost:27017/smsdb')
  .then(() => console.log('MongoDB connected ✅'))
  .catch(err => console.error('DB Error:', err));

// ===== ROUTES =====
const studentRoutes = require('./routes/students');
app.use('/api/students', studentRoutes);

// Auth routes
const authRoutes = require('./routes/auth');
app.use('/api/auth', authRoutes);
```

```
// Root route
app.get('/', (req, res) => {
  res.json({ message: 'SMS API v1.0', status: 'Running' });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

// Error handler middleware (4 params = error handler)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal Server Error' });
});

// ===== START SERVER =====
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Middleware — Viva Critical Concept

```
// Middleware = function with (req, res, next)
// It sits BETWEEN request and response

// 1. Application-level middleware (runs on every request)
app.use((req, res, next) => {
  console.log('Request received');
  next(); // Pass to next middleware/route
});

// 2. Route-level middleware (runs on specific route)
function isAuthenticated(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  // Verify token...
  next();
}

// Apply to protected route
app.get('/api/profile', isAuthenticated, (req, res) => {
```

```
res.json({ message: 'Protected data' });
});

// 3. Built-in middleware
app.use(express.json());           // Parse JSON bodies
app.use(express.static('public')); // Serve static files
```

Middleware Flow: Request → Logger MW → Auth MW → Route Handler → Response

SECTION 3 — MONGODB + MONGOOSE

Exam Definition

MongoDB is a **document-oriented NoSQL database** that stores data as **BSON documents** (Binary JSON) in collections instead of rows in tables.

SQL vs NoSQL — MOST ASKED COMPARISON

Feature	SQL (MySQL)	NoSQL (MongoDB)
Data Format	Tables with rows/columns	Collections with JSON documents
Schema	Fixed, predefined	Dynamic, flexible
Relationships	JOINS across tables	Embedded documents or references
Scaling	Vertical (bigger server)	Horizontal (more servers)
Query Language	SQL	MongoDB Query Language (MQL)
Transactions	ACID-compliant	BASE (eventual consistency)
Best For	Structured, relational data	Unstructured, hierarchical data

One-liner: "SQL = rigid spreadsheets with JOINS. MongoDB = flexible JSON files in folders (collections)."

Mongoose Schema + Model

```
// models/Student.js
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name too short']
  },
  email: {
```

```

    type: String,
    required: true,
    unique: true,
    lowercase: true,
    match: [/^\S+@\S+\.\S+/, 'Invalid email format']
  },
  course: {
    type: String,
    enum: ['IT', 'CS', 'ENTC', 'Mechanical'],
    default: 'IT'
  },
  rollNo: {
    type: Number,
    unique: true
  },
  marks: [Number], // Array of numbers
  registeredAt: {
    type: Date,
    default: Date.now
  },
  isActive: {
    type: Boolean,
    default: true
  }
});

// Virtual field (computed, not stored in DB)
studentSchema.virtual('average').get(function() {
  if (!this.marks.length) return 0;
  return this.marks.reduce((s, m) => s + m, 0) / this.marks.length;
});

module.exports = mongoose.model('Student', studentSchema);
// MongoDB creates collection: 'students' (pluralized, lowercase)

```

Complete CRUD Routes

```

// routes/students.js
const express = require('express');
const router = express.Router();
const Student = require('../models/Student');

// ===== CREATE - POST /api/students =====

```

```
router.post('/', async (req, res) => {
  try {
    const student = new Student(req.body);
    const saved = await student.save();
    res.status(201).json({ success: true, data: saved });
  } catch (err) {
    if (err.code === 11000) {
      // MongoDB duplicate key error
      return res.status(400).json({ error: 'Email already exists' });
    }
    res.status(400).json({ error: err.message });
  }
});

// ===== READ ALL - GET /api/students =====
router.get('/', async (req, res) => {
  try {
    const { course, search } = req.query;
    let filter = {};

    if (course) filter.course = course;
    if (search) filter.name = { $regex: search, $options: 'i' };

    const students = await Student.find(filter).sort({ registeredAt: -1 });
    res.json({ success: true, count: students.length, data: students });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== READ ONE - GET /api/students/:id =====
router.get('/:id', async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) {
      return res.status(404).json({ error: 'Student not found' });
    }
    res.json({ success: true, data: student });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== UPDATE - PUT /api/students/:id =====
```

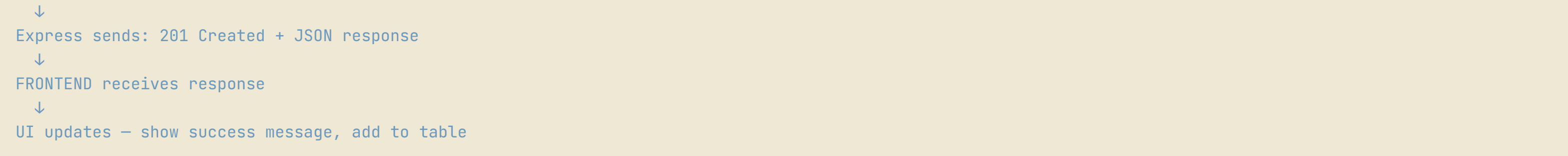
```
router.put('/:id', async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true } // new=return updated doc
    );
    if (!student) return res.status(404).json({ error: 'Not found' });
    res.json({ success: true, data: student });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// ===== DELETE - DELETE /api/students/:id =====
router.delete('/:id', async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ error: 'Not found' });
    res.json({ success: true, message: 'Student deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

Full CRUD Flow Diagram (Viva Answer)

```
FRONTEND (Angular/HTML)
  | User clicks "Register"
  ↓
AJAX / HTTP POST → http://localhost:3000/api/students
  ↓
EXPRESS MIDDLEWARE (cors, json parser, logger)
  ↓
ROUTE HANDLER (POST /api/students)
  ↓
MONGOOSE (validate + save)
  ↓
MONGODB (stores document in 'students' collection)
  ↓
MongoDB returns saved document
```



Viva Quote: "Our frontend sends a POST request with student JSON. Express receives it, the middleware parses the body, the route handler creates a Mongoose model instance, calls `.save()`, and MongoDB stores it. We respond with status 201 and the saved document. The Angular component's `.subscribe()` receives the response and updates the table."

SECTION 4 — AUTHENTICATION

Session vs JWT Authentication

Feature	Session Auth	JWT Auth
Storage	Server-side session store	Client-side token (localStorage)
Stateful?	Stateful (server remembers)	Stateless (server forgets)
Scalability	Hard (server sticky sessions)	Easy (any server can verify)
Logout	Delete session from server	Token expires or blacklisted
Used In	Traditional MVC apps	REST APIs, SPAs

Basic HTTP Auth in Express

```
// Simple Basic Auth middleware
function basicAuth(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader) return res.status(401).json({ error: 'No auth header' });

  // "Basic base64(username:password)"
  const base64 = authHeader.split(' ')[1];
  const [username, password] = Buffer.from(base64, 'base64').toString().split(':');

  if (username === 'admin' && password === 'password123') {
    next();
  } else {
    res.status(403).json({ error: 'Invalid credentials' });
  }
}
```

```
app.get('/api/admin', basicAuth, (req, res) => {  
  res.json({ message: 'Admin area' });  
});
```

VIVA Q&A — Unit 4

Q1: What is Node.js? Why use it for backend?

Node.js is a JavaScript runtime built on V8 engine. Use it for backend because: same language (JS) for frontend + backend, non-blocking I/O handles many concurrent requests efficiently, huge NPM ecosystem, great for real-time applications.

Q2: What is middleware in Express?

Middleware is a function with `(req, res, next)` parameters that runs between receiving a request and sending a response. Uses: logging, authentication, body parsing, error handling. Must call `next()` to pass control to the next middleware, or send a response to end the chain.

Q3: What is REST API? Explain CRUD with HTTP methods.

REST API uses HTTP methods to perform operations: **GET** = Read, **POST** = Create, **PUT** = Update, **DELETE** = Remove. It's stateless — each request is independent. URLs represent resources: `/api/students` = all students, `/api/students/:id` = specific student.

Q4: What is Mongoose? Why use it with MongoDB?

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides: Schema definition (structure for documents), built-in validation, middleware hooks (pre/post save), and a clean API for CRUD operations. MongoDB is schema-less, but Mongoose adds structure and safety.

Q5: What is the difference between `find()` and `findById()` ?

`find(filter)` returns an array of documents matching the filter — used to get all/many. `findById(id)` returns a single document by its MongoDB `_id` — used to get one specific record.

Oral Trap Questions

Trap: "What is the `_id` field in MongoDB?"

MongoDB automatically adds `_id` to every document — it's a unique `ObjectId` (12-byte BSON type). It's automatically indexed. Never set `_id` manually unless you have a specific reason.

Trap: "What happens if you don't call `next()` in middleware?"

The request hangs forever — the client receives no response and eventually times out. Always call `next()` to pass control, or send a `res.json()` / `res.send()` to end the request-response cycle.

Assignment 3 — Practical Viva Flow

"I created a Node.js + Express REST API with 4 endpoints for CRUD operations on student data. MongoDB is used as the database with Mongoose ODM for schema validation. The 4 APIs are: **POST /api/students** (create), **GET /api/students** (read all), **PUT /api/students/:id** (update), **DELETE /api/students/:id** (delete). For static file hosting, I used `express.static('public')` middleware."

High Probability Questions

1. What is Node.js? How is it different from other server-side languages?
2. Explain non-blocking I/O with Event Loop
3. What is Express.js? Why use it?
4. Explain middleware with example
5. What is REST API? HTTP methods for CRUD?
6. SQL vs NoSQL — when to use which?
7. What is Mongoose? Schema and Model?
8. Write a POST route in Express that saves to MongoDB
9. What is `async/await` in Node.js?
10. What is the full CRUD flow (frontend to database)?